Name :- Prathamesh Arvind Jadhav Roll No:- 4059

Experiment No.5

- **Title:-** Implement Bag-of-Words (Count Occurrence, Normalized Count Occurrence), TF-IDF, and Word2Vec Embeddings.

- **Aim:-** To Implement Bag-of-Words (Count Occurrence, Normalized Count Occurrence), TF-IDF, and Word2Vec Embeddings.

- **Objective:-**

1. Understand the concept of Bag-of-Words (BoW) and its limitations.
2. Implement Bag-of-Words (count occurrence and normalized form) using Python.
3. Apply TF-IDF to assign importance weights to words.
4. Generate dense vector representations using Word2Vec embeddings.
5. Compare sparse vs. dense representations of text data.

- **Procedure and Flowchart:-**

Step 1: Install required Python libraries:

pip install scikit-learn gensim

Step 2: Import necessary modules from sklearn and gensim.

Step 3: Define a sample text corpus.

Step 4: Apply Bag-of-Words (Count Occurrence).

Step 5: Apply Normalized Bag-of-Words (Term Frequency).

Step 6: Apply TF-IDF transformation.

Step 7: Train Word2Vec model on the text data.

Step 8: Display results for each method and analyze differences.

Name :- Prathamesh Arvind Jadhav Roll No:- 4059

- **Python code:-**

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
import matplotlib.pyplot as plt
import seaborn as sns
# Sample data (replace with your dataset)
corpus = [
    "Natural language processing enables computers to understand human language",
    "Word embeddings like Word2Vec represent words in vector space",
    "Bag of words and TF-IDF are traditional NLP approaches",
    "Deep learning models improve performance in NLP tasks"
]
# 1. Bag-of-Words (Count Occurrence)
count_vectorizer = CountVectorizer()
count_matrix = count_vectorizer.fit_transform(corpus)
print("Vocabulary:", count_vectorizer.get_feature_names_out())
print("\nBag-of-Words (Count Occurrence):")
print(pd.DataFrame(count_matrix.toarray(),
columns=count_vectorizer.get_feature_names_out()))
# Visualization: Bag-of-Words
plt.figure(figsize=(8, 4))
sns.heatmap(pd.DataFrame(count_matrix.toarray(),
columns=count_vectorizer.get_feature_names_out()), annot=True, cmap="Blues")
plt.title("Bag-of-Words (Count Occurrence)")
```

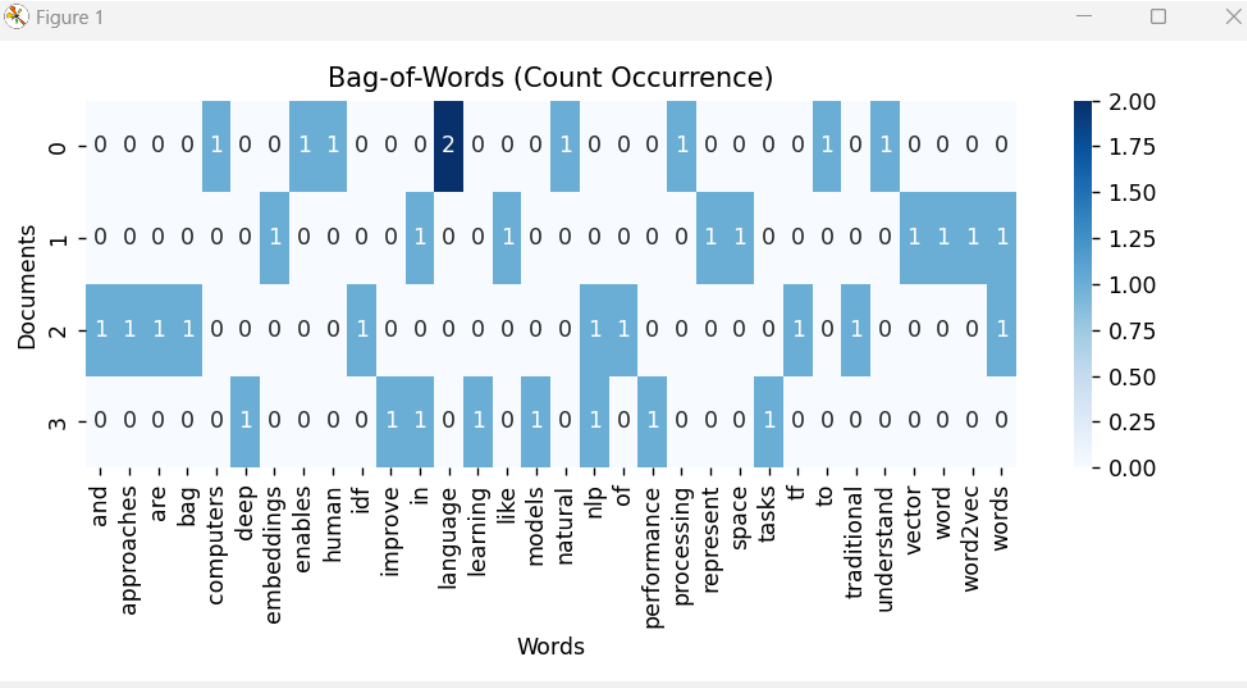
```
plt.xlabel("Words")

plt.ylabel("Documents")

plt.tight_layout()

plt.show()

# 2. Normalized Count Occurrence (Frequency)

normalized_count = count_matrix.toarray().astype(float)

normalized_count = normalized_count / normalized_count.sum(axis=1, keepdims=True)

print("\nNormalized Count Occurrence (Term Frequency):")

print(pd.DataFrame(normalized_count, columns=count_vectorizer.get_feature_names_out()))

# Visualization: Normalized Count Occurrence

plt.figure(figsize=(8, 4))

sns.heatmap(pd.DataFrame(normalized_count,
columns=count_vectorizer.get_feature_names_out()), annot=True, cmap="Greens")

plt.title("Normalized Count Occurrence (Term Frequency)")

plt.xlabel("Words")

plt.ylabel("Documents")

plt.tight_layout()

plt.show()

# 3. TF-IDF

tfidf_vectorizer = TfidfVectorizer()

tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)

print("\nTF-IDF Matrix:")

print(pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf_vectorizer.get_feature_names_out()))

# Visualization: TF-IDF

plt.figure(figsize=(8, 4))

sns.heatmap(pd.DataFrame(tfidf_matrix.toarray(),
columns=tfidf_vectorizer.get_feature_names_out()), annot=True, cmap="Purples")
```

```
plt.title("TF-IDF Matrix")

plt.xlabel("Words")

plt.ylabel("Documents")

plt.tight_layout()

plt.show()

# 4. Word2Vec Embeddings

# Tokenize corpus

tokenized_corpus = [word_tokenize(doc.lower()) for doc in corpus]

# Train Word2Vec

w2v_model = Word2Vec(sentences=tokenized_corpus, vector_size=50, window=5,
min_count=1, workers=4)

# Example: Get embedding of a word

word = "language"

print(f"\nWord2Vec embedding for '{word}':\n", w2v_model.wv[word])

# Example: Find most similar words

print("\nMost similar words to 'language':", w2v_model.wv.most_similar("language"))
```

- **Output:-**

```
PS D:\NLP\Exp5> python Exp5.py
Vocabulary: ['and' 'approaches' 'are' 'bag' 'computers' 'deep' 'embeddings' 'enables'
 'human' 'idf' 'improve' 'in' 'language' 'learning' 'like' 'models'
 'natural' 'nlp' 'of' 'performance' 'processing' 'represent' 'space'
 'tasks' 'tf' 'to' 'traditional' 'understand' 'vector' 'word' 'word2vec'
 'words']

Bag-of-Words (Count Occurrence):
   and  approaches  are  bag  computers  deep  ...  traditional  understand  vector  word  word2vec
  words
0    0           0    0    0          1     0  ...            0           1       0     0         0
     0
1    0           0    0    0          0     0  ...            0           0       1     1         1
     1
2    1           1    1    1          0     0  ...            1           0       0     0         0
     1
3    0           0    0    0          0     1  ...            0           0       0     0         0
     0

[4 rows x 32 columns]

Normalized Count Occurrence (Term Frequency):
   and  approaches  are  bag  computers   deep  ...  traditional  understand    vector      word  w
ord2vec     words
0  0.0         0.0  0.0  0.0   0.111111  0.000  ...          0.0    0.111111  0.000000  0.000000  0
.000000  0.000000
1  0.0         0.0  0.0  0.0   0.000000  0.000  ...          0.0    0.000000  0.111111  0.111111  0
.111111  0.111111
2  0.1         0.1  0.1  0.1   0.000000  0.000  ...          0.1    0.000000  0.000000  0.000000  0
.000000  0.100000
3  0.0         0.0  0.0  0.0   0.000000  0.125  ...          0.0    0.000000  0.000000  0.000000  0
.000000  0.000000

[4 rows x 32 columns]
```

```
TF-IDF Matrix:
        and  approaches       are       bag  computers  ...  understand    vector      word  word2v
ec     words
0  0.000000    0.000000  0.000000  0.000000   0.301511  ...    0.301511  0.000000  0.000000  0.0000
00  0.000000
1  0.000000    0.000000  0.000000  0.000000   0.000000  ...    0.000000  0.348299  0.348299  0.3482
99  0.274603
2  0.328919    0.328919  0.328919  0.328919   0.000000  ...    0.000000  0.000000  0.000000  0.0000
00  0.259324
3  0.000000    0.000000  0.000000  0.000000   0.000000  ...    0.000000  0.000000  0.000000  0.0000
00  0.000000

[4 rows x 32 columns]

Word2Vec embedding for 'language':
 [-0.01631583  0.0089916  -0.00827415  0.00164907  0.01699724 -0.00892435
  0.009035   -0.01357392 -0.00709698  0.01879702 -0.00315531  0.00064274
 -0.00828126 -0.01536538 -0.00301602  0.00493959 -0.00177605  0.01106732
 -0.00548595  0.00452013  0.01091159  0.01669191 -0.00290748 -0.01841629
  0.0087411   0.00114357  0.01488382 -0.00162657 -0.00527683 -0.01750602
 -0.00171311  0.00565313  0.01080286  0.01410531 -0.01140624  0.00371764
  0.01217773 -0.0095961  -0.00621452  0.01359526  0.00326295  0.00037983
  0.00694727  0.00043555  0.01923765  0.01012121 -0.01783478 -0.01408312
  0.00180291  0.01278507]

Most similar words to 'language': [('natural', 0.37571486830711365), ('vector', 0.22977666556835175
), ('models', 0.22082354128360748), ('approaches', 0.21908226609230042), ('bag', 0.1608981937170028
7), ('space', 0.14872416853904724), ('tasks', 0.12489066272974014), ('embeddings', 0.08060287684202
194), ('enables', 0.07399576157331467), ('and', 0.05545045807957649)]
```



- **Question and Answer:-**

Q1. What is the difference between Bag-of-Words and TF-IDF?

Ans->

Aspect	**Bag-of-Words (BoW)**	**TF-IDF**
Weighting	Pure word counts/frequencies	Weighted by importance (TF × IDF)
Common words effect	High frequency → high importance (even for stopwords like *the*)	Penalized (low IDF)
Captures importance?	No	Yes
Example use cases	Simple text classification, spam detection	Information retrieval, search engines, NLP tasks needing meaningful features

Q2. Why is normalization important in Bag-of-Words representation?

Ans->

Normalization in the Bag-of-Words (BoW) representation is important because it removes the bias caused by differences in document length. In BoW, documents are represented using raw word counts, which means longer documents naturally produce higher values even if the relative importance of words is the same as in shorter documents. Without normalization, this leads to unfair comparisons, where algorithms may mistakenly give more weight to longer texts. By normalizing, we scale the word counts so that documents can be compared on the basis of word distribution rather than size. For example, L1 normalization converts counts into relative frequencies, while L2 normalization ensures the vector length is 1, making cosine similarity and other distance-based measures more meaningful. In short, normalization makes the BoW features consistent, comparable, and more useful for machine learning models.

Q3. Explain how Word2Vec captures semantic meaning compared to BoW?

Ans->

Bag-of-Words (BoW)

- **How it works:** Represents text as a sparse vector of word counts or frequencies.
- **Problem:**
    - Words are treated as independent; no relationship between them.
    - "King" and "Queen" are as unrelated as "King" and "Table."
    - Order and context of words are lost.
- **Meaning captured?** No real semantic understanding—only presence/absence or frequency.

Word2Vec

- **How it works:** Learns **dense vector representations** (embeddings) of words using neural networks.
- **Key idea:** Words that appear in **similar contexts** have similar meanings.
- **Two main models:**

- **CBOW (Continuous Bag-of-Words):** Predicts a word from surrounding context.
- **Skip-Gram:** Predicts surrounding context from a given word.

- **Why it captures meaning:**
  - By training on large text corpora, Word2Vec places semantically similar words closer in vector space.

Q4. What are the advantages of Word2Vec over TF-IDF?

Ans->

- **Advantages of Word2Vec over TF-IDF**

1. **Captures Semantic Meaning**
   - Word2Vec places semantically similar words near each other in vector space.
   - TF-IDF treats each word as independent, so synonyms aren't recognized.
2. **Dense Representation**
   - Word2Vec produces low-dimensional vectors (e.g., 100–300 dimensions), making computations faster and more memory-efficient.
   - TF-IDF creates very high-dimensional sparse vectors (one dimension per vocabulary word).
3. **Handles Synonyms & Context**
   - Word2Vec recognizes that *"good"* and *"excellent"* are similar in meaning.
   - TF-IDF cannot — both get totally separate vector positions.
4. **Word Relationships & Analogies**
   - Word2Vec captures relationships like *"Paris - France + Italy = Rome"*.
   - TF-IDF has no way of encoding such relationships.
5. **Better for Downstream Tasks**
   - Word2Vec embeddings improve performance in tasks like sentiment analysis, machine translation, or recommendation systems.
   - TF-IDF is mostly useful for keyword-based tasks like search or text classification.

6. **Generalization Beyond Exact Words**
    - Word2Vec can relate unseen but contextually similar words if trained on enough data.

• Conclusion:-

This experiment demonstrates how text can be represented numerically using Bag-of-Words, TF-IDF, and Word2Vec embeddings. While BoW and TF-IDF provide sparse representations, Word2Vec generates dense semantic embeddings that capture contextual meaning.